

Svelte **Review Cards**



Eight concepts an interviewer can probe and a beginner must own.
Read the question, cover the answer, answer aloud, then check the
code and the figure.

1 — Prototype: own first, then inherited

OBJECTS LOOKUP INHERITANCE

Q. You never added `publish` to `piece`, so where does `piece.publish()` come from?

ANSWER. Every object carries a hidden link to another object called its prototype. When you read a property, JavaScript looks on the object itself first; if the property is not there, it follows that hidden link up to the prototype and uses whatever it finds. `publish` was never put on `piece`, so the lookup walks up to `Article.prototype` and finds it there.

WHY IT WORKS. A property stored directly on the object is an own property, returned at once. A property found by walking up the chain is inherited. There is only one shared copy of `publish` on `Article.prototype`, but every instance can reach it, so a million `Article` objects share one method instead of each carrying its own.

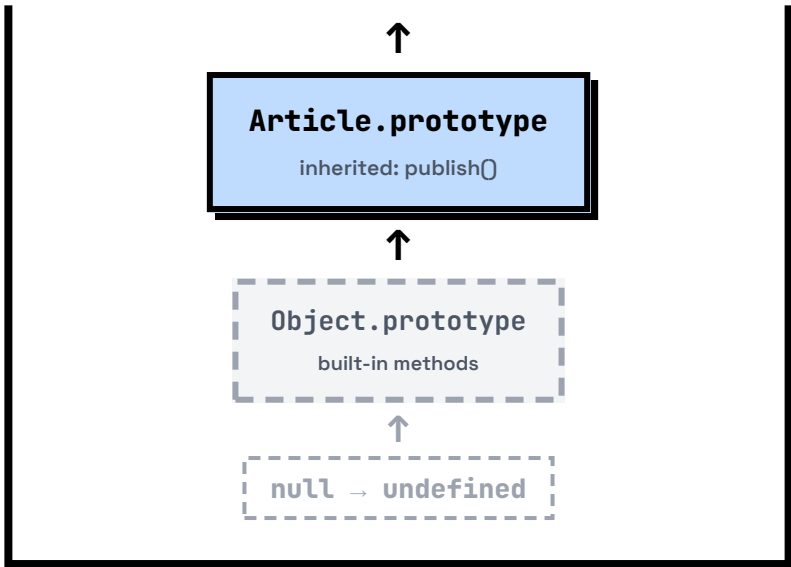
examples/Article.js

```
class Article {  
  publish() { return "published"; } // on  
  Article.prototype  
}  
const piece = new Article(); // a new  
instance  
piece.headline = "Markets rally"; // an own  
property  
console.log(piece.headline); // "Markets  
rally" – own  
console.log(piece.publish()); // "published"  
– inherited  
console.log(piece.author); // undefined –  
not on the chain
```

THE LOOKUP WALKS UP

piece

own: headline = "Markets rally"



- ① **Summary.** A read checks own properties first, then walks up the prototype chain until it finds a match or hits `null`. Today a read only *finds* a value; the next idea turns a read into something that can *run* code.

2 — Closure: a function that keeps its variables alive

SCOPE PRIVACY STATE

Q. `createClapButton()` has returned, so why can `article.total()` still read `claps`?

ANSWER. A function's local variables normally vanish when it returns. But if the function hands back something that still references those variables, they stay alive. The bundle of a function plus the variables it captured is a closure, and the captured `claps` lives on, reachable only through `clap` and `total`.

WHY IT WORKS. Both `clap` and `total` were declared inside `createClapButton`, so they close over the *same* `claps`. When the factory returns, those two methods still hold it, so JavaScript keeps the variable in memory. Nothing outside can name `claps` directly, which makes it private state you can only touch through the methods.

examples/clapButton.js

```
function createClapButton() {  
  let claps = 0; // private,  
  starts at 0  
  return {  
    clap() { claps++; }, // increments  
    the captured claps  
    total() { return claps; } // reads the  
    captured claps  
  };  
}  
const article = createClapButton(); // the scope  
stays alive  
article.clap(); // claps → 1  
article.clap(); // claps → 2  
console.log(article.total()); // prints 2  
console.log(article.claps); // undefined –  
sealed inside
```

ONE HIDDEN VARIABLE

```
let claps = 0
```

PRIVATE · SHARED · ALIVE



```
clap()
```

WRITES CLAPS



```
total()
```

READS CLAPS

- ① **Summary.** A closure keeps a function's variables alive after it returns, shared between the methods that captured them and unreachable from anywhere else. Private state is powerful, but it is trapped inside one function — what if many files needed the same value?

3 — Module: state created once, shared by every importer

IMPORTS SCOPE SINGLETON

Q. Two files both import { current } from session.js. Do they each get their own currentUser?

ANSWER. No. A module's top-level code runs exactly once, the first time anything imports it. So `currentUser` is created once, and every file that imports `session.js` reaches that single shared value, never a private copy.

WHY IT WORKS. The variable is never exported, so it is private to the module; the only way in or out is through the exported `login` and `current` functions. That makes the module a closure stretched over a whole file. One shared instance, touched only through its public functions, lives for the whole program. The pattern has a name: a singleton.

```
let currentUser = "John";           // private
module state

export function login(name) {        // writes
  currentUser = name;
}

export function current() {          // reads
  return currentUser;
}
```

examples/session.js

```
import { login, current } from "../session.js";

console.log(current());              // "John"
// (the initial value)
login("Ada Lovelace");               // changes
// the shared value
console.log(current());              // "Ada
// Lovelace" (seen everywhere)
```

examples/app.js

ONE SHARED STORE

session.js

created ONCE

```
let currentUser = "John"
```

login()

current()

THE ONLY DOORS

↓ import

app.js reaches the SAME value

- ① **Summary.** A module runs once, so any top-level variable becomes a shared singleton, reached only through the functions it exports. The value is shared now, but reads and writes are still plain — what if a *read* itself could run code?

4 — Getter: a read that secretly runs a function

OBJECTS ACCESSOR INTERCEPTION

Q. `article.byline` has no parentheses, so why does it return a freshly built string every time you read it?

ANSWER. Because `byline` is not a stored value. It is a getter: a property defined with the `get` keyword that secretly runs a function on every read. To the call site it looks identical to plain data, but inside, code is executing.

WHY IT WORKS. Reading `article.author` is a plain data-property access — JavaScript hands back the stored string and nothing runs. Reading `article.byline` looks the same, but the `get` keyword makes it an accessor: reading it runs `get byline()` and returns whatever the function produces. Because it recomputes from the current `author` on every access, it can never go stale. This is the first crack in the passive read.

```
const article = {  
  author: "Ada Lovelace",  
  data property  
  get byline() {  
    runs on every read  
    return "By " + this.author;  
    fresh value  
  }  
};  
console.log(article.byline);  
Lovelace" (no parentheses)  
article.author = "Grace Hopper";  
underlying data  
console.log(article.byline);  
Hopper" (recomputed)
```

examples/byline.js

TWO READS, TWO OUTCOMES

`article.author`

→

stored string

DATA PROPERTY · NO CODE RUNS

RUNS CODE

article.byline → **runs get byline()**

ACCESSOR · RECOMPUTES EVERY READ

- ① **Summary.** A getter is a function disguised as a property: reading it runs code and returns a fresh value, so it can never go stale. The catch is that a getter guards one property you named in advance — what catches a property you didn't name?

5 — Proxy: traps that fire on every read and write

OBJECTS TRAPS INTERCEPTION

Q. `claps` was never defined on the object, so why does `article.claps = 5` get logged?

ANSWER. Because `article` is not the real object. It is a proxy wrapping a target, with a handler that intercepts every operation. Writing `article.claps = 5` fires the `set` trap before anything reaches the target, and the trap logs the access, even for a property that did not exist a moment ago.

WHY IT WORKS. A getter guards one property you named in advance. A proxy sits in front of the *entire* object. Every read runs the `get` trap and every write runs the `set` trap, no matter which key is touched, no matter whether it existed before. Your code runs in the gap between the access and the target, every single time.

examples/proxy.js

```
const target = { headline: "Markets rally" };
const article = new Proxy(target, { // wraps
  target in a proxy
    get(t, k) { // get trap -
      runs on every read
        console.log("read:", k);
        return t[k];
      },
    set(t, k, v) { // set trap -
      runs on every write
        console.log("write:", k);
        t[k] = v;
        return true;
      }
  });
article.headline; // fires get
→ "read: headline"
article.claps = 5; // fires set
→ "write: claps"
```

THE CHECKPOINT

1 · WRITE

```
article.claps = 5
```



2 · TRAP

```
set(t, "claps", 5)
```

LOGS "WRITE: CLAPS"



3 · TARGET

```
target.claps = 5
```

A BRAND-NEW PROPERTY

- ① **Summary.** A proxy intercepts every read and write through traps, even on properties that did not exist a moment ago. We can now run code on every access — the open question is what the most useful thing to run there would be.

6 — Observer: track on read, trigger on write

REACTIVITY TRACK TRIGGER

Q. An effect reads `claps()` exactly once. Later, `setClaps(2)` re-runs it. How did the write know the effect cared?

ANSWER. Because the read remembered. While an effect runs, a single global slot called `activeEffect` holds it. When the effect reads a signal, that signal records the effect in its `deps` set — that is *track on read*. When the signal is later written, it walks that set and re-runs every recorded effect — that is *trigger on write*.

WHY IT WORKS. `effect(fn)` parks `fn` in the slot, then runs it once so its reads can subscribe. Each signal it touches sees the slot occupied and adds `fn` to its dependents. The slot is cleared, but the subscriptions remain. A later write re-runs exactly those dependents and nobody else. Two moves — track and trigger — and you have reactivity.

examples/observer.js

```
let activeEffect = null; // who is
reading right now?
function signal(initial) {
  let value = initial;
  const deps = new Set();
  function read() {
    if (activeEffect) deps.add(activeEffect); //
    track on read
    return value;
  }
  function write(next) {
    value = next;
    deps.forEach(run => run()); //
    trigger on write
  }
  return [read, write];
}
function effect(fn) {
  activeEffect = fn; // park fn as
  the active reader
  fn(); // run once so
  its reads subscribe
```

```
    activeEffect = null;
  }
  const [claps, setClaps] = signal(0);
  effect(() => console.log("claps:", claps())); //
  prints "claps: 0"
  setClaps(1); //
  re-runs effect → "claps: 1"
  setClaps(2); //
  re-runs effect → "claps: 2"
```

THE ENGINE, TWO MOVES

A · TRACK

effect reads claps() → **signal records fn**

deps ← **fn** · parked in **activeEffect** while reading

B · TRIGGER

setClaps(2) writes → **deps re-run fn()**

"claps: 2" · every recorded effect, no one else

- ① **Summary.** A value that notices its readers and re-runs them when it changes is the whole engine of reactivity — track on read, trigger on write. The next step is to package that engine so plain dot-notation does both moves for you.

7 — Reactive object: reads track, writes trigger, through plain dots

PROXY OBSERVER REACTIVITY

Q. With `reactive({ claps: 0 })`, you never call `subscribe`. So why does `article.claps = 2` re-run the effect that read it?

ANSWER. Because the object is wrapped in a proxy whose traps are wired to the track-and-trigger engine. Reading `article.claps` runs the `get` trap, which records the active effect. Writing `article.claps = 2` runs the `set` trap, which re-runs every recorded effect. You wrote plain dot syntax; the proxy did the bookkeeping.

WHY IT WORKS. The pieces from the last two cards are joined here. The proxy intercepts every property, even ones added later. The `get` trap uses that interception to *track* whoever is reading. The `set` trap uses it to *trigger* everyone who ever read. The surface stays clean — `article.claps`, no method calls — and underneath, an entire dependency graph keeps itself in sync.

```
let activeEffect = null; // who's
reading right now?
function reactive(target) {
  const deps = new Set();
  return new Proxy(target, { // wrap in a
    proxy
      get(obj, key) {
        if (activeEffect) deps.add(activeEffect); //
        track on read
        return obj[key];
      },
      set(obj, key, value) {
        obj[key] = value;
        deps.forEach(run => run()); // trigger on
        write
        return true;
      }
    });
}
function effect(fn) {
  activeEffect = fn;
  fn();
}
```

examples/reactive.js

```
activeEffect = null;
}  
const article = reactive({ claps: 0 });  
effect(() => console.log("claps:", article.claps));  
// "claps: 0"  
article.claps = 1;  
// "claps: 1"  
article.claps = 2;  
// "claps: 2"
```

ONE SURFACE, TWO FACES

article.claps = 2

CLEAN SYNTAX · NO SUBSCRIBE CALL



get TRAP

on read

TRACKS THE READER

set TRAP

on write

TRIGGERS READERS

- ① **Summary.** A reactive object hides the track-and-trigger engine behind ordinary dot-notation: writes trigger, reads track, all through proxy traps. This is almost exactly what a framework hands you, and what Svelte calls a rune.

8 — State to DOM: only the node that read the value re-runs

SVELTE RUNES FINE-GRAINED

Q. In a button that renders `{claps}` claps, clicking it updates just the number and nothing else on the page. How?

ANSWER. Because the `{claps}` text node is the effect. Reading `claps` inside the markup subscribes that one DOM node to the reactive value. Clicking writes `claps`, the write triggers, and Svelte re-runs only the nodes that read it — a single text update, not a re-render of the component.

WHY IT WORKS. `$state(0)` is the reactive object from the previous step, handed to you as a rune. `{claps}` is the read that subscribes the text node. `claps++` is the write that triggers. So the dependent — and only the dependent — re-runs. Unlike a virtual-DOM framework, where a state change re-runs the whole component function and a diff decides what changed, Svelte tracks the dependency at the node and re-runs just that node.

routes/+page.svelte

```
<script>
  let claps = $state(0);           // a reactive
  value — a signal
</script>

<button onclick={() => claps++}>    // writes
  claps (trigger half)
  {claps} claps                    // reads claps
  (this text node subscribes)
</button>
```

TWO WAYS TO UPDATE

VIRTUAL DOM

`claps++` → re-run whole component → build new tree → diff vs old tree → patch

SVELTE · SURGICAL

`$state(0)`

→

`{claps} node`

`claps++` → only this text node updates

- ① **Summary.** `$state` is the engine we built by hand: the text node that read `claps` is the effect, the click is the trigger, and only that node re-runs. No magic was left — every piece was plain JavaScript.